

TP : Prédiction des Espèces d'Iris avec Machine Learning

⚠ Ici ce n'est **pas** une régression, c'est un problème de classification.

PARTIE 1 : Chargement du Dataset

Étape 1 : Importation des bibliothèques

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.datasets import load_iris
```

Étape 2 : Charger les données

```
data = load_iris()

X = data.data
y = data.target

print("Nombre d'échantillons :", X.shape[0])
print("Nombre de variables :", X.shape[1])
```

Explication

Le dataset contient :

- **150 fleurs**
- **4 variables**

Les variables sont :

- longueur du sépale
- largeur du sépale

- longueur du pétale
- largeur du pétale

Les classes :

- Setosa
 - Versicolor
 - Virginica
-

PARTIE 2 : Exploration des données

```
df = pd.DataFrame(X, columns=data.feature_names)
```

```
df["target"] = y
```

```
print(df.head())  
print(df.info())  
print(df.describe())
```

Objectif

Comprendre :

- les variables
 - les statistiques
 - la structure du dataset
-

Visualisation

```
plt.figure()  
sns.pairplot(df, hue="target")
```

```
plt.show()
```

Explication

Cette visualisation montre **la relation entre les variables et les différentes espèces de fleurs**.

PARTIE 3 : Séparation Train/Test

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.3,  
    random_state=42  
)
```

Explication

- 70% entraînement
 - 30% test
-

PARTIE 4 : Création du Modèle

Nous utilisons **KNN (K-Nearest Neighbors)**.

```
from sklearn.neighbors import KNeighborsClassifier
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
model.fit(X_train, y_train)
```

Explication

Le modèle classe une fleur selon **les fleurs les plus proches**.

PARTIE 5 : Prédiction

```
y_pred = model.predict(X_test)
```

Le modèle prédit maintenant **l'espèce de la fleur**.

PARTIE 6 : Évaluation du Modèle

```
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy :", accuracy)
```

```
print("Matrice de confusion :")  
print(confusion_matrix(y_test, y_pred))
```

Explication

Accuracy

→ pourcentage de bonnes prédictions.

Matrice de confusion

→ compare **valeurs réelles** et **valeurs prédites**.

PARTIE 7 : Visualisation de la Matrice de Confusion

```
plt.figure()
```

```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, cmap="Blues")
```

```
plt.title("Matrice de confusion")
```

```
plt.show()
```